实验三 代码

```
%{

#include <stdio.h>
#define PROGRAM      1
#define VAR          2
#define INTEGER      3
#define REAL         4
#define BEGINN       5
#define END          6
#define IF           7
#define THEN         8
#define ELSE         9
#define WHILE        10
#define DO           11
#define ID           12
#define INT          13
#define FLOAT        14
#define COMMA        15
#define COLON        16
#define SEMICOLON    17
#define LPAREN       18
#define RPAREN       19
#define PERIOD       20
#define ASSIGN       21
#define EQ           22
#define NE           23
#define LT           24
#define LE           25
#define GT           26
#define GE           27
#define PLUS         28
#define MINUS        29
#define TIMES        30
#define DIVIDE       31
#define ERRORCHAR    32
#define NEWLINE      33

%}


delim       [ \t \n]
ws          {delim}+
letter      [A-Za-z]
digit       [0-9]
```

```
id          {letter}({letter}|{digit})*
/*number        {digit}+(\.{digit}+)?(E[+-]?{digit}+)?*/
/* 整数：一个或多个数字 */
int_num     {digit}+
/* 浮点数：可以是 .12 或 12.34 */
float_num   (\.{digit}+)|({digit}+\.{digit}+)

/* 状态（或条件）定义可以定义在这里
 * INITIAL是一个默认的状态，不需要定义
 */
%s COMMENT

%%

<INITIAL>"/*"             {BEGIN COMMENT;}
<COMMENT>"*/"             {BEGIN INITIAL;}
<COMMENT>.|\n             {;}

<INITIAL>{ws}             {;}

<INITIAL>"PROGRAM"      { return PROGRAM; }
<INITIAL>"VAR"          { return VAR; }
<INITIAL>"INTEGER"      { return INTEGER; }
<INITIAL>"REAL"         { return REAL; }
<INITIAL>"BEGIN"        { return BEGINN; }
<INITIAL>"END"          { return END; }
<INITIAL>"IF"           { return IF; }
<INITIAL>"THEN"         { return THEN; }
<INITIAL>"ELSE"         { return ELSE; }
<INITIAL>"WHILE"        { return WHILE; }
<INITIAL>"DO"           { return DO; }

<INITIAL>{id}           { return ID; }
<INITIAL>{float_num}    { return FLOAT; }
<INITIAL>{int_num}      { return INT; }

<INITIAL>":="           { return ASSIGN; }
<INITIAL>":"            { return COLON; }
<INITIAL>","            { return COMMA; }
<INITIAL>";"            { return SEMICOLON; }
<INITIAL>"("            { return LPAREN; }
<INITIAL>")"            { return RPAREN; }
<INITIAL>"."            { return PERIOD; }
<INITIAL>"="            { return EQ; }
<INITIAL>"<>"           { return NE; }
<INITIAL>"<="           { return LE; }
<INITIAL>"<"            { return LT; }
```

```
<INITIAL>">="           { return GE; }
<INITIAL>">"            { return GT; }
<INITIAL>"+"            { return PLUS; }
<INITIAL>"-"            { return MINUS; }
<INITIAL>"*"            { return TIMES; }
<INITIAL>"/"            { return DIVIDE; }

<INITIAL>.                {return ERRORCHAR;}


%%

int yywrap (){
  return 1;
}

void writeout(int c){
  switch(c){
    case PROGRAM: fprintf(yyout, "(PROGRAM, \"%s\") ", yytext); break;
    case VAR: fprintf(yyout, "(VAR, \"%s\") ", yytext); break;
    case INTEGER: fprintf(yyout, "(INTEGER, \"%s\") ", yytext); break;
    case REAL: fprintf(yyout, "(REAL, \"%s\") ", yytext); break;
    case BEGINN: fprintf(yyout, "(BEGIN, \"%s\") ", yytext); break;
    case END: fprintf(yyout, "(END, \"%s\") ", yytext); break;
    case IF: fprintf(yyout, "(IF, \"%s\") ", yytext); break;
    case THEN: fprintf(yyout, "(THEN, \"%s\") ", yytext); break;
    case ELSE: fprintf(yyout, "(ELSE, \"%s\") ", yytext); break;
    case WHILE: fprintf(yyout, "(WHILE, \"%s\") ", yytext); break;
    case DO: fprintf(yyout, "(DO, \"%s\") ", yytext); break;
    case ID: fprintf(yyout, "(ID, \"%s\") ", yytext); break;
    case INT: fprintf(yyout, "(INT, \"%s\") ", yytext); break;
    case FLOAT: fprintf(yyout, "(FLOAT, \"%s\") ", yytext); break;
    case COMMA: fprintf(yyout, "(COMMA, \"%s\") ", yytext); break;
    case COLON: fprintf(yyout, "(COLON, \"%s\") ", yytext); break;
    case SEMICOLON: fprintf(yyout, "(SEMICOLON, \"%s\") ", yytext); break;
    case LPAREN: fprintf(yyout, "(LPAREN, \"%s\") ", yytext); break;
    case RPAREN: fprintf(yyout, "(RPAREN, \"%s\") ", yytext); break;
    case PERIOD: fprintf(yyout, "(PERIOD, \"%s\") ", yytext); break;
    case ASSIGN: fprintf(yyout, "(ASSIGN, \"%s\") ", yytext); break;
    case EQ: fprintf(yyout, "(EQ, \"%s\") ", yytext); break;
    case NE: fprintf(yyout, "(NE, \"%s\") ", yytext); break;
    case LT: fprintf(yyout, "(LT, \"%s\") ", yytext); break;
    case LE: fprintf(yyout, "(LE, \"%s\") ", yytext); break;
    case GT: fprintf(yyout, "(GT, \"%s\") ", yytext); break;
    case GE: fprintf(yyout, "(GE, \"%s\") ", yytext); break;
    case PLUS: fprintf(yyout, "(PLUS, \"%s\") ", yytext); break;
    case MINUS: fprintf(yyout, "(MINUS, \"%s\") ", yytext); break;
```

```
    case TIMES: fprintf(yyout, "(TIMES, \"%s\") ", yytext); break;
    case DIVIDE: fprintf(yyout, "(DIVIDE, \"%s\") ", yytext); break;
    case ERRORCHAR: fprintf(yyout, "(ERRORCHAR, \"%s\") ", yytext); break;
    case NEWLINE: fprintf(yyout, "\n"); break;
    default:break;
  }
  return;
}


int main (int argc, char ** argv){
    int c,j=0;
    if (argc>=2){
      if ((yyin = fopen(argv[1], "r")) == NULL){
        printf("Can't open file %s\n", argv[1]);
        return 1;
      }
      if (argc>=3){
        yyout=fopen(argv[2], "w");
      }
    }

    while (c = yylex()){
        writeout(c);
        j++;
        if (j%5 == 0) writeout(NEWLINE);
    }
    if(argc>=2){
      fclose(yyin);
      if (argc>=3) fclose(yyout);
    }
    return 0;
}

```

实验四 代码

```
%{

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

```

```
#define PROGRAM      1
#define VAR          2
#define INTEGER      3
#define REAL         4
#define BEGINN       5
#define END          6
#define IF           7
#define THEN         8
#define ELSE         9
#define WHILE        10
#define DO           11
#define ID           12
#define INT          13
#define FLOAT        14
#define COMMA        15
#define COLON        16
#define SEMICOLON    17
#define LPAREN       18
#define RPAREN       19
#define PERIOD       20
#define ASSIGN       21
#define EQ           22
#define NE           23
#define LT           24
#define LE           25
#define GT           26
#define GE           27
#define PLUS         28
#define MINUS        29
#define TIMES        30
#define DIVIDE       31
#define ERRORCHAR    32
#define NEWLINE      33


    /* 提示：你需要将yylval的类型修改为union类型!
    * 对于记号ID来说，属性值是ID的名字（字符串）；
    * 对于整数和浮点数来说，属性值是整数或者浮点数的值。
    */
typedef union
{
    int ival;
    float fval;
    char *sval;
}YYTYPE;

YYTYPE yylval;
```

```
    /* yylval 是yacc中定义的变量，用来保存记号的属性值，默认是int类型。
    * 在用lex实现的词法分析器中可以使用这个变量将记号的属性传递给用
    * yacc实现的语法分析器。
    */

%}


delim       [ \t \n]
ws          {delim}+
letter  [A-Za-z]
digit       [0-9]
id          {letter}({letter}|{digit})*
int_num     {digit}+
float_num   ({digit}+\.{digit}*)|(\.{digit}+)

/* 状态（或条件）定义可以定义在这里
 * INITIAL是一个默认的状态，不需要定义
 */
%s COMMENT

%%

<INITIAL>"/*"                       {BEGIN COMMENT;}
<COMMENT>"*/"                       {BEGIN INITIAL;}
<COMMENT>.|\n                       {;}

 /* ECHO是一个宏，相当于 fprintf(yyout, "%s", yytext)*/

<INITIAL>{ws}             {;}
<INITIAL>"PROGRAM"      { yylval.sval = strdup(yytext); return PROGRAM; }
<INITIAL>"VAR"          { yylval.sval = strdup(yytext); return VAR; }
<INITIAL>"INTEGER"      { yylval.sval = strdup(yytext); return INTEGER; }
<INITIAL>"REAL"         { yylval.sval = strdup(yytext); return REAL; }
<INITIAL>"BEGIN"        { yylval.sval = strdup(yytext); return BEGINN; }
<INITIAL>"END"          { yylval.sval = strdup(yytext); return END; }
<INITIAL>"IF"           { yylval.sval = strdup(yytext); return IF; }
<INITIAL>"THEN"         { yylval.sval = strdup(yytext); return THEN; }
<INITIAL>"ELSE"         { yylval.sval = strdup(yytext); return ELSE; }
<INITIAL>"WHILE"        { yylval.sval = strdup(yytext); return WHILE; }
<INITIAL>"DO"           { yylval.sval = strdup(yytext); return DO; }

<INITIAL>{id}           { yylval.sval = strdup(yytext); return ID; }
<INITIAL>{float_num}    { yylval.fval = atof(yytext); return FLOAT; }
<INITIAL>{int_num}      { yylval.ival = atoi(yytext); return INT; }

<INITIAL>":="           { yylval.sval = strdup(yytext); return ASSIGN; }
```

```
<INITIAL>":"            { yylval.sval = strdup(yytext); return COLON; }
<INITIAL>","            { yylval.sval = strdup(yytext); return COMMA; }
<INITIAL>";"            { yylval.sval = strdup(yytext); return SEMICOLON; }
<INITIAL>"("            { yylval.sval = strdup(yytext); return LPAREN; }
<INITIAL>")"            { yylval.sval = strdup(yytext); return RPAREN; }
<INITIAL>"."            { yylval.sval = strdup(yytext); return PERIOD; }
<INITIAL>"="            { yylval.sval = strdup(yytext); return EQ; }
<INITIAL>"<>"           { yylval.sval = strdup(yytext); return NE; }
<INITIAL>"<="           { yylval.sval = strdup(yytext); return LE; }
<INITIAL>"<"            { yylval.sval = strdup(yytext); return LT; }
<INITIAL>">="           { yylval.sval = strdup(yytext); return GE; }
<INITIAL>">"            { yylval.sval = strdup(yytext); return GT; }
<INITIAL>"+"            { yylval.sval = strdup(yytext); return PLUS; }
<INITIAL>"-"            { yylval.sval = strdup(yytext); return MINUS; }
<INITIAL>"*"            { yylval.sval = strdup(yytext); return TIMES; }
<INITIAL>"/"            { yylval.sval = strdup(yytext); return DIVIDE; }

<INITIAL>.              { return ERRORCHAR; }


%%

int yywrap (){
  return 1;
}

    /* 请你修改该函数，利用yylval来输出字符串、整数、浮点数的值
    * 枚举类型可以保持不变，继续用yytext输出
    */
void writeout(int c){
    switch(c) {
        case PROGRAM: fprintf(yyout, "(PROGRAM, \"%s\") ", yylval.sval);
break;
        case VAR: fprintf(yyout, "(VAR, \"%s\") ", yylval.sval); break;
        case INTEGER: fprintf(yyout, "(INTEGER, \"%s\") ", yylval.sval);
break;
        case REAL: fprintf(yyout, "(REAL, \"%s\") ", yylval.sval); break;
        case BEGINN: fprintf(yyout, "(BEGIN, \"%s\") ", yylval.sval); break;
        case END: fprintf(yyout, "(END, \"%s\") ", yylval.sval); break;
        case IF: fprintf(yyout, "(IF, \"%s\") ", yylval.sval); break;
        case THEN: fprintf(yyout, "(THEN, \"%s\") ", yylval.sval); break;
        case ELSE: fprintf(yyout, "(ELSE, \"%s\") ", yylval.sval); break;
        case WHILE: fprintf(yyout, "(WHILE, \"%s\") ", yylval.sval); break;
        case DO: fprintf(yyout, "(DO, \"%s\") ", yylval.sval); break;
        case ID: fprintf(yyout, "(ID, \"%s\") ", yylval.sval); break;
        case INT: fprintf(yyout, "(INT, \"%d\") ", yylval.ival); break;
        case FLOAT: fprintf(yyout, "(FLOAT, \"%f\") ", yylval.fval); break;
```

```
146         case COMMA: fprintf(yyout, "(COMMA, \"%s\") ", yylval.sval); break;
147         case COLON: fprintf(yyout, "(COLON, \"%s\") ", yylval.sval); break;
148         case SEMICOLON: fprintf(yyout, "(SEMICOLON, \"%s\") ", yylval.sval);
    break;
149         case LPAREN: fprintf(yyout, "(LPAREN, \"%s\") ", yylval.sval); break;
150         case RPAREN: fprintf(yyout, "(RPAREN, \"%s\") ", yylval.sval); break;
151         case PERIOD: fprintf(yyout, "(PERIOD, \"%s\") ", yylval.sval); break;
152         case ASSIGN: fprintf(yyout, "(ASSIGN, \"%s\") ", yylval.sval); break;
153         case EQ: fprintf(yyout, "(EQ, \"%s\") ", yylval.sval); break;
154         case NE: fprintf(yyout, "(NE, \"%s\") ", yylval.sval); break;
155         case LT: fprintf(yyout, "(LT, \"%s\") ", yylval.sval); break;
156         case LE: fprintf(yyout, "(LE, \"%s\") ", yylval.sval); break;
157         case GT: fprintf(yyout, "(GT, \"%s\") ", yylval.sval); break;
158         case GE: fprintf(yyout, "(GE, \"%s\") ", yylval.sval); break;
159         case PLUS: fprintf(yyout, "(PLUS, \"%s\") ", yylval.sval); break;
160         case MINUS: fprintf(yyout, "(MINUS, \"%s\") ", yylval.sval); break;
161         case TIMES: fprintf(yyout, "(TIMES, \"%s\") ", yylval.sval); break;
162         case DIVIDE: fprintf(yyout, "(DIVIDE, \"%s\") ", yylval.sval); break;
163         case ERRORCHAR: fprintf(yyout, "(ERRORCHAR, \"%s\") ", yytext); break;
164         case NEWLINE: fprintf(yyout, "\n"); break;
165         default: break;
166     }
167   return;
168 }
169
170
171 int main (int argc, char ** argv){
172     int c,j=0;
173     if (argc>=2){
174       if ((yyin = fopen(argv[1], "r")) == NULL){
175         printf("Can't open file %s\n", argv[1]);
176         return 1;
177       }
178       if (argc>=3){
179         yyout=fopen(argv[2], "w");
180       }
181     }
182
183     while (c = yylex()){
184         writeout(c);
185         j++;
186         if (j%5 == 0) writeout(NEWLINE);
187     }
188     if(argc>=2){
189       fclose(yyin);
190       if (argc>=3) fclose(yyout);
191     }
```

```
192     return 0;
193 }
194
```

实验四 调试问题

1、自测运行时发现输出了注释



原因：在注释段落，编译器初始代码添加了ECHO，忘记删除，导致注释虽然被识别了但是照常输出。

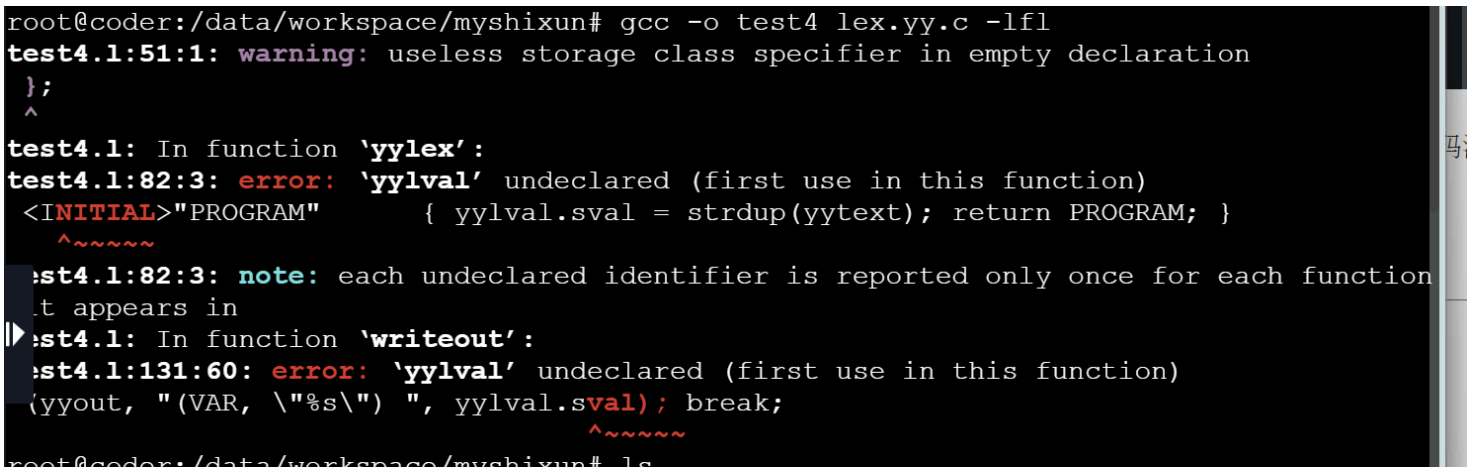
👉 修改后代码如下：

```
%s COMMENT

%%

<INITIAL>"/*"                   {BEGIN COMMENT;}
<COMMENT>"*/"                   {BEGIN INITIAL;}
<COMMENT>.|\n                   {;}
```

2、在编译文件时出现如下报错，显示yylval未定义



原因：检查代码后发现yylval的union类型定义有问题

```
typedef union yylval
{
    int ival;
    float fval;
    char *sval;
};
```

👉 修改代码后如下：

```
typedef union
{
    int ival;
    float fval;
    char *sval;
}YYTYPE;

YYTYPE yylval;
```